



Tài Liệu Kiến Trúc Hệ Thống: NexOS Cloud

Tài liệu này mô tả chi tiết các công nghệ hiện đang được sử dụng để xây dựng Web OS (NexOS), cũng như kiến trúc tổng thể về Frontend, Backend, Lưu trữ (Storage) và Hệ quản trị cơ sở dữ liệu (Database).

1. Công nghệ ứng dụng

Hệ thống NexOS sử dụng cấu trúc **Full-stack JavaScript (Node.js & Vanilla JS)** nguyên bản giúp tối ưu hóa luồng dữ liệu, tăng hiệu suất tức thời và giảm độ phụ thuộc vào các thư viện cộng kênh.

- **Frontend:** HTML5, CSS3 (Vanilla CSS) và Vanilla JavaScript.
- **Backend:** Node.js kết hợp cùng framework Express.js.
- **Giao tiếp Real-time:** công nghệ WebSockets qua thư viện [Socket.io](#).
- **Cơ sở dữ liệu:** Dữ liệu quan hệ nhẹ gọn với SQLite.
- **DevOps / Triển khai:** Phân phối và đóng gói môi trường qua Docker và Docker Compose.

2. Kiến trúc Frontend (Trình diễn giao diện người dùng)

Frontend của NexOS được mô phỏng hoàn chỉnh như một hệ điều hành Desktop ngay trên trình duyệt máy tính dưới dạng **Single Page Application (SPA)**.

- **Window Manager (Quản lý cửa sổ):** Hệ thống xây dựng nhân đồ họa riêng cho phép mở và render nhiều "cửa sổ ứng dụng" cùng một lúc (như File Explorer, Text Editor, Cloud Drive,...). Hỗ trợ thao tác kéo thả, thu phóng và sắp xếp chiều sâu (z-index) giống hệ điều hành thật.
- **Virtual File System (Hệ điều hành tập tin ảo):** Trình duyệt tự duy trì một cây thư mục ảo kết nối xuyên suốt với dữ liệu trả về từ server, cho phép hiển thị và truy xuất các thư mục `/documents` , `/shared` ,...

- **Tích hợp Cloud Native:** App "Cloud Drive" kết nối trực tiếp với backend, duy trì trạng thái lưu trữ ảo với các tab điều hướng rõ rệt như `My Drive`, `Shared`.

3. Kiến trúc Backend (Máy chủ xử lý API)

Backend đóng vai trò là "Não bộ" (Core Server), đứng đằng sau xử lý mọi logic theo kiến trúc RESTful kết hợp Realtime.

- **User Authentication (Authentication & Authorization):** Middleware trên máy chủ thiết lập cơ chế kiểm tra `x-user` ở mọi endpoint. Tầng này tra cứu CSDL để xác định người thao tác là ai và chặn các hành vi trái phép.
- **Role-based Access Control (Kiểm soát phân quyền - RBAC):** Dữ liệu phân tách rõ ràng người dùng bình thường và `Administrator / Cloud Operator`. Tài khoản Admin có đặc quyền xem toàn cục `SELECT * FROM metadata` và xóa mọi file trên hệ thống.
- **Socket Real-time Sync:** Nhờ WebSockets, ngay khi một file được upload, đổi tên hoặc xóa thành công, Backend lập tức gửi sự kiện broadcast (phát sóng) đến mọi client đang kết nối. Khách hàng bên kia sẽ tự động refresh giao diện mà không cần F5 trình duyệt.

4. Kiến trúc Lưu trữ (Storage Layer)

Mô đun Storage của hệ thống không dùng cách lưu file bình thường mà được thiết kế theo dạng **Abstraction** cao, mở đường cho việc "Scale" (mở rộng) lên Server Cloud chuyên nghiệp (ví dụ AWS S3) trong tương lai:

- **Physical Blob Storage (Lưu trữ vật lý làm phẳng):** Các file tải lên từ người dùng (ảnh, text) không được lưu theo cấu trúc thư mục lồng nhau. Chúng được mã hóa bằng thuật toán băm cơ bản (Base64 file Id) và lưu "phẳng" (không phân cấp) tại thư mục `cloud_data/`.
- **Docker Persistent Volume (Khả năng lưu trữ vĩnh viễn):** Toàn bộ thư mục ảo `cloud_data/` chạy trong môi trường cục bộ của Docker Container nhưng được ràng buộc thẳng ra ổ đĩa máy Host môi trường Windows của bạn (qua Bind Mount `./cloud_data:/app/cloud_data`). Ngay cả khi server sập, update, thao tác rebuild đi nữa, toàn bộ dữ liệu người dùng vẫn còn nguyên vẹn và an toàn.

5. Cơ sở dữ liệu (Database Architecture)

Thay vì sử dụng JSON dạng phẳng (flat file) như trước, hệ thống đã trang bị **SQLite3** chuyên nghiệp để đáp ứng những lệnh truy vấn phức tạp của nhiều người thao tác cùng lúc:

- **Nguyên tắc đóng gói:** File vật lý `database.sqlite` được đặt ngay bên trong `cloud_data/` để đảm bảo database và file vật lý lúc nào cũng đi kèm với nhau, loại bỏ nguy cơ mất thông tin đồng bộ.
- **Cấu trúc Bảng `users`** : Quản lý thông tin tài khoản với PK (khóa chính) là `username` , chứa `password` và phân quyền `role` .
- **Cấu trúc Bảng `metadata`** : Đây là **bộ não ánh xạ** quan trọng nhất thay thế cho cấu trúc cây thư mục kiểu cũ. Nó gồm `id` , `owner` , `path` ảo (đường dẫn mà giả lập trên Front-end hiển thị), `type` , và `fileId` (mã ánh xạ đến file vật lý). Với thiết kế tách lớp (decoupling) này, nếu người dùng đổi tên một thư mục lớn, backend chỉ việc sửa chuỗi `path` ở Metadata trên database, mà không cần tốn công bê vác hay copy-paste hàng Gigabytes bản thân các file vật lý bên trong.

Tài liệu này được soạn thảo vào lúc Hệ thống Web OS vừa hoàn thành giai đoạn Containerization triển khai mạnh mẽ với cơ chế phân quyền chia sẻ (Shared Filesystem) hoàn chỉnh.